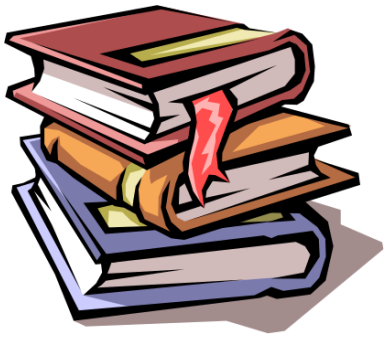
FPT UNIVERSITY

Assignment Report

ASSIGNMENT-01 – HOW TO WRITE A DEVICE DRIVER

ioctl interface · misc device · ARMv7 target (QEMU vexpress-a9)



Students	Phạm Võ Hoàng Nhân Nguyễn Duy Khoa Ngô Từ Nhã Quyên Nguyễn Cao Kỳ
Class	IC2008
Lecturer	Phạm Thế Vinh
Date	17/10/2026

1. Overview

This assignment adds two kernel modules to the Linux 5.15 system built in the labs, running on QEMU `vexpress-a9` (Cortex-A9, ARMv7-A) and cross-compiled with `arm-linux-gnueabihf-gcc` 15.2.

`asgn1_driver` (Task 1) is a character driver registered with `register_chrdev()` on major 241. It keeps a 1024-byte buffer, exposes it through `/dev/asgn1`, and adds four ioctl commands: `ASGN1_RESET_BUFFER`, `ASGN1_GET_STATS`, `ASGN1_SET_MODE` and `ASGN1_GET_VERSION`. It also publishes its state in `/proc/asgn1_info` and in three sysfs attributes. Because a shell cannot issue an arbitrary ioctl, a small user program, `asgn1_ioctl_test`, opens the device and calls `ioctl()` with the commands from the shared header `asgn1_ioctl.h`.

`counter_driver` (Task 2) is a separate module written from scratch on the miscdevice framework. `/dev/counter` returns the current value on each read and then increments it; writing a non-negative decimal number sets the counter; `/proc/counter_info` shows the value and how many reads and writes have happened since the module was loaded.

Both modules were tested inside QEMU with the scripts `test_asgn1.sh` and `test_counter.sh`; the complete logs are in section 5.

2. Architecture of the two drivers

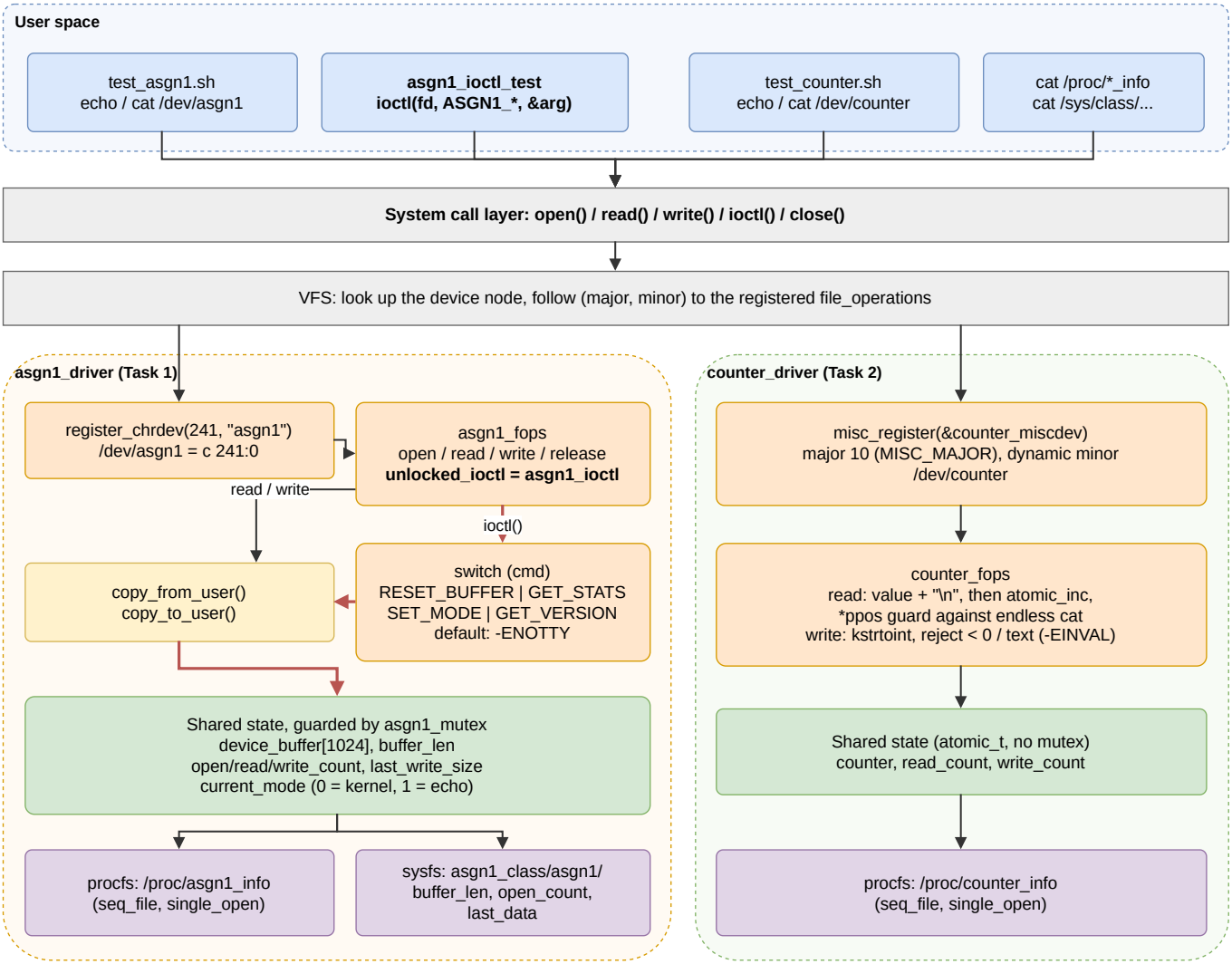


Figure 1: Both drivers from user space down to their state and proc/sys entries. The red path is the ioctl path of Task 1.

Every request starts as a system call in user space. VFS looks at the device node, takes its (major, minor) pair and finds the `file_operations` table registered for it. For `/dev/asgn1` this is `asgn1_fops`, found through major 241; for `/dev/counter` it is `counter_fops`, found through major 10 and the minor that the misc core handed out. From there each driver works only on its own state: Task 1 keeps a buffer and several plain `int` counters behind one mutex, Task 2 keeps three `atomic_t` values and needs no lock. The proc and sys entries never go through the device node; they read the same state directly whenever someone runs `cat`.

The ioctl path (red in Figure 1, detailed in Figure 2) is the main addition of Task 1. `ioctl(fd, cmd, arg)` enters the kernel, VFS calls `fops->unlocked_ioctl`, which is `asgn1_ioctl()`. The driver switches on `cmd`; for commands that carry data it moves the data with `copy_from_user()` (`SET_MODE`) or `copy_to_user()` (`GET_STATS`, `GET_VERSION`), and returns 0, `-EFAULT` when the user pointer is bad, `-EINVAL` for a mode other than 0 or 1, or `-ENOTTY` for an unknown command.

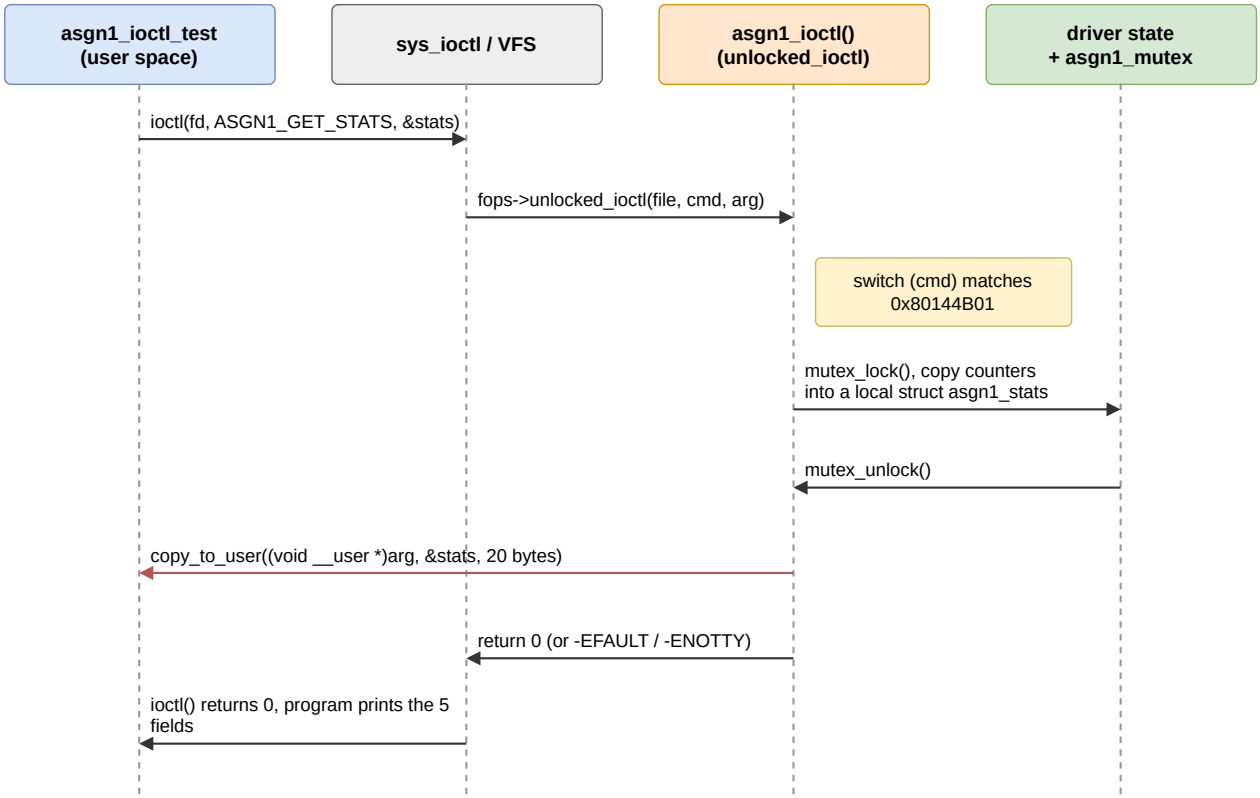


Figure 2: Sequence of one `ASGN1_GET_STATS` call

The statistics are copied into a local `struct asgn1_stats` while the mutex is held, and the copy to user space happens after unlocking. This keeps the lock short and, more importantly, means the driver never sleeps in `copy_to_user()` while holding the lock that every `read()` and `write()` also needs.

3. Character, block and network drivers

	Character device	Block device	Network device
Interface to the kernel	`struct file_operations` (open, read, write, unlocked_ioctl, mmap…), reached through a node in `/dev`	`struct block_device_operations` (open, release, ioctl, getgeo) plus a request queue (blk-mq) that carries the actual I/O	`struct net_device_ops` (ndo_open, ndo_stop, ndo_start_xmit…); no node in `/dev`, reached through sockets and an interface name such as `eth0`
Unit of transfer	Byte stream of any length	Fixed-size blocks / sectors (512 B or more), addressed by sector number	Packets (frames), pushed by the device as they arrive
Buffering	None imposed; the driver decides (our 1024-byte buffer, or none at all for the counter)	Page cache above it and an I/O scheduler that merges and reorders requests	`sk_buff` queues: TX queue with qdisc, RX via NAPI polling
Embedded examples	UART (`ttyAMA0`), GPIO, I^2C/SPI sensors, `/dev/counter`	eMMC, SD card (`mmcblk0`), USB stick, `mtdblock` on top of flash	Ethernet MAC (`smsc911x` on vexpress), Wi-Fi, CAN (SocketCAN)

Two design decisions show how the choice works in practice. A temperature sensor on I^2C produces a few bytes on request and has nothing to seek or cache, so it belongs to a character driver (or the IIO subsystem built on the same idea); making it a block device would force it to look like a disk with sectors. An SD card, on the other hand, stores a filesystem that is read and written in blocks at random positions and benefits from the page cache and request merging, so it must be a block device; a character driver would leave every filesystem unable to mount it. A third case, a CAN controller, carries frames to many applications at once, which is exactly what the socket layer of a network driver offers, so SocketCAN exposes it as a network interface rather than as `/dev/can0`.

4. Design analysis

register_chrdev() in Task 1, misc_register() in Task 2

`register_chrdev(241, "asgn1", &asgn1_fops)` claims major 241 and all 256 minors under it for a single device. The driver also has to build its place in the device model itself: `class_create()` for `/sys/class/asgn1_class`, `device_create()` for the device, `sysfs_create_group()` for the attributes, and the matching destroy calls in reverse order in `asgn1_exit()`. The test script still runs `mknod /dev/asgn1 c 241 0` when the node is missing. The advantages are a fixed, known major (useful when the node is created by a script or baked into a read-only rootfs) and a class of our own to hang attributes on. The disadvantages are that a fixed number may collide with another driver, a whole major is spent on one device, and there is more code to get wrong in the error paths.

`misc_register(&counter_miscdev)` with `MISC_DYNAMIC_MINOR` does all of that in one call: the device shares major 10 with every other misc device, gets a free minor, appears in `/sys/class/misc/counter`, and devtmpfs creates `/dev/counter` automatically. Removing it is one call to `misc_deregister()`. The price is less control: the major is always 10, the

minor is not known in advance, and there is only one device per registration.

Our rule of thumb from doing both: a single, simple device such as a counter, a watchdog or a control node fits the misc framework; a driver that serves several instances, needs a stable major, or wants its own class (Task 1 with its custom sysfs group) is better served by `register_chrdev()` or, in newer code, `alloc_chrdev_region()` with `cdev`.

Why Task 2 uses atomic_t

`counter++` on a plain `int` compiles to three steps on ARM: load the value, add one, store it back. On our kernel two CPUs are online (the boot log shows "SMP: Total of 2 processors activated"), so two `cat /dev/counter` running at the same time can both load 5, both add one and both store 6: two reads, but the counter only moved by one and `read_count` also lost a count. Even on one CPU, preemption between the load and the store gives the same result. `atomic_inc()` turns the read-modify-write into one indivisible operation (on ARMv7 an `ldrex`/`strex` loop that retries if another CPU touched the word in between), so no increment is ever lost and no mutex is needed.

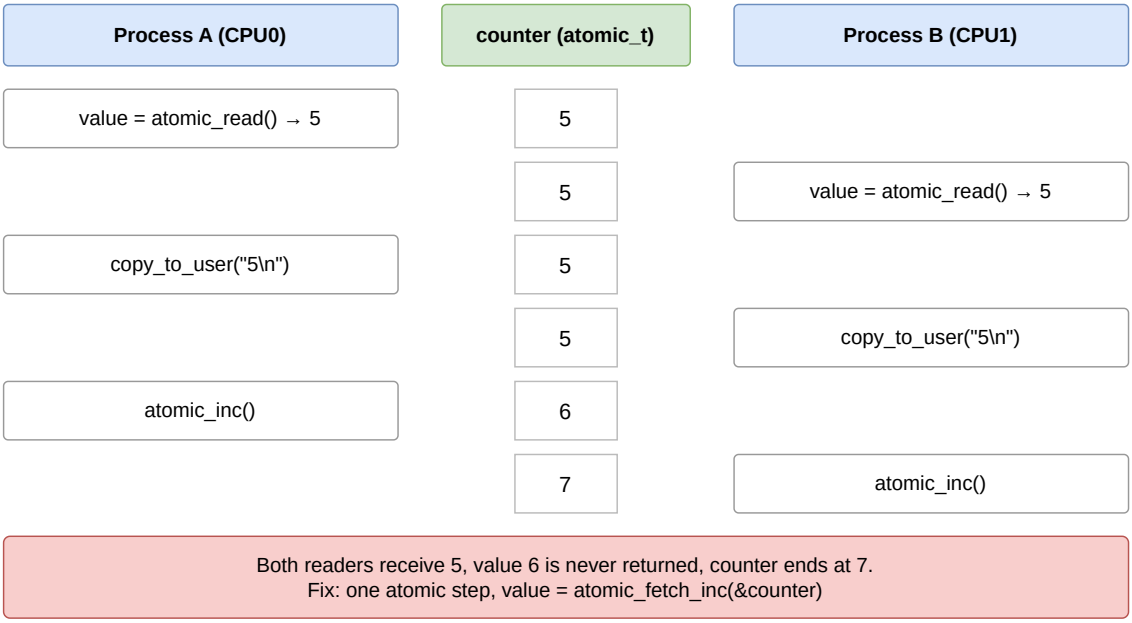


Figure 3: A race that remains in `counter_read()`: read and increment are two separate atomic steps

While reviewing the code we found that `atomic_t` alone does not make the whole read correct. `counter_read()` calls `atomic_read()`, copies the value to the user, and only then calls `atomic_inc()`. Each step is atomic, but the pair is not. If two processes read at the same moment (Figure 3), both can see 5, both return `"5\n"`, and the counter ends at 7, so the value 6 is never handed out. The same gap lets a write slip in: a reader that has already loaded 5 increments after a concurrent `echo 100`, turning it into 101 without anyone having read 100. The fix is to make taking the value and incrementing one operation, `value = atomic_fetch_inc(&counter)` (or `atomic_inc_return() - 1`), and only then format and copy the value. Our single-process test script cannot trigger this case, which is why all its results are correct.

The ioctl magic number

An ioctl command is a 32-bit number, and every driver receives whatever number user space passes. If two drivers both used plain 0, 1, 2, 3, a program that opens the wrong node by mistake could trigger a completely unrelated action. The `_IO`/`_IOR`/`_IOW` macros avoid this by packing four fields into

the number: the direction of data transfer, the size of the argument, a "type" byte (the magic number, unique per driver family) and the command number. Our header uses the magic `'K'` (0x4B):

```
#define ASGN1_MAGIC 'K'
#define ASGN1_VERSION_LEN 64
#define ASGN1_RESET_BUFFER _IO(ASGN1_MAGIC, 0)
#define ASGN1_GET_STATS _IOR(ASGN1_MAGIC, 1, struct asgn1_stats)
#define ASGN1_SET_MODE _IOW(ASGN1_MAGIC, 2, int)
#define ASGN1_GET_VERSION _IOR(ASGN1_MAGIC, 3, char[ASGN1_VERSION_LEN])
```

Command	dir bits 31-30	size bits 29-16	type (magic) bits 15-8	nr bits 7-0	cmd value
ASGN1_RESET_BUFFER _IO('K', 0)	00 (none)	0	0x4B 'K'	0	**0x00004B00**
ASGN1_GET_STATS _IOR('K', 1, struct asgn1_stats)	10 (read)	20	0x4B 'K'	1	**0x80144B01**
ASGN1_SET_MODE _IOW('K', 2, int)	01 (write)	4	0x4B 'K'	2	**0x40044B02**
ASGN1_GET_VERSION _IOR('K', 3, char[64])	10 (read)	64	0x4B 'K'	3	**0x80404B03**

Figure 4: Bit layout of the four commands on ARM

On ARM the layout is `dir` in bits 31–30 (01 = write, 10 = read), `size` in bits 29–16, `type` in bits 15–8 and `nr` in bits 7–0. Taking `ASGN1_GET_STATS` as an example: `struct asgn1_stats` has five `int` fields, so its size is 20 bytes (0x14), and the value is $2 \cdot 2^{30} + 20 \cdot 2^{16} + \texttt{0x4B} \cdot 2^{8} + 1 = \texttt{0x80144B01}$. The kernel helpers simply undo the packing: `_IOC_NR(cmd) = cmd & 0xFF = 1`, `_IOC_TYPE(cmd) = (cmd >> 8) & 0xFF = 0x4B`, `_IOC_SIZE(cmd) = (cmd >> 16) & 0x3FFF = 20` and `_IOC_DIR(cmd) = cmd >> 30 = 2`. A driver can use these to reject a command whose type byte is not its own before even reaching the `switch`, and the size field documents how many bytes `copy_to_user()` is expected to move. Ours relies on the `switch` on the full value and returns `-ENOTTY` in the `default` branch, which gives the same protection because the full value already includes the magic.

5. Test results

Task 1: test_asgn1.sh

The script loads the module, creates `/dev/asgn1` if needed, then exercises the four ioctl commands in the order `GET_VERSION`, `SET_MODE 1` with a write and read, `SET_MODE 0` with a write and read, `GET_STATS`, `RESET_BUFFER`, a read after reset and `GET_STATS` again, and finally unloads the module and prints the kernel log. The complete output:

```
/ # test_asgn1.sh
===== ASSIGNMENT 01 - TASK 1 TEST =====
asgn1_driver: loading out-of-tree module taints kernel.
asgn1_driver: initializing module
asgn1_driver: loaded, major=241
asgn1_driver: /proc/asgn1_info created
```

```
[1] GET_VERSION
asgn1_driver: device opened (count=1)
asgn1: GET_VERSION called
VERSION: version v1 SE200494
asgn1_driver: device closed
[2] ECHO MODE
asgn1_driver: device opened (count=2)
asgn1: SET_MODE called
 SET MODE: done
asgn1_driver: device closed
asgn1_driver: device opened (count=3)
asgn1_driver: received 17 bytes: [hello-normal mode]
asgn1_driver: device closed
asgn1_driver: device opened (count=4)
asgn1_driver: sent 17 bytes to user
asgn1_driver: device closed
read = hello-normal mode
[3] KERNEL MODE
asgn1_driver: device opened (count=5)
asgn1: SET_MODE called
 SET MODE: done
asgn1_driver: device closed
asgn1_driver: device opened (count=6)
asgn1_driver: received 12 bytes: [[KERNEL] hello-kernel]
asgn1_driver: device closed
asgn1_driver: device opened (count=7)
asgn1_driver: sent 21 bytes to user
asgn1_driver: device closed
read = [[KERNEL] hello-kernel]
[4] GET_STATS
asgn1_driver: device opened (count=8)
asgn1: GET_STATS called
 OPEN COUNT:        8
 WRITE COUNT:       2
 READ COUNT:        2
 BUFFER LEN:        21
 LAST WRITE SIZE:   12
 GET_STATS: done
asgn1_driver: device closed
[5] RESET_BUFFER
asgn1_driver: device opened (count=9)
asgn1: RESET_BUFFER called
 RESET: done
asgn1_driver: device closed
asgn1_driver: device opened (count=10)
asgn1_driver: device closed
after reset = []
asgn1_driver: device opened (count=11)
asgn1: GET_STATS called
 OPEN COUNT:        11
 WRITE COUNT:       0
 READ COUNT:        0
 BUFFER LEN:        0
 LAST WRITE SIZE:   12
 GET_STATS: done
asgn1_driver: device closed
[6] UNLOAD MODULE
asgn1_driver: module unloaded
[7] DMESG
asgn1_driver: loading out-of-tree module taints kernel.
```

```
asgn1_driver: initializing module
asgn1_driver: loaded, major=241
asgn1_driver: /proc/asgn1_info created
asgn1_driver: device opened (count=1)
asgn1: GET_VERSION called
asgn1_driver: device closed
asgn1_driver: device opened (count=2)
asgn1: SET_MODE called
asgn1_driver: device closed
asgn1_driver: device opened (count=3)
asgn1_driver: received 17 bytes: [hello-normal mode]
asgn1_driver: device closed
asgn1_driver: device opened (count=4)
asgn1_driver: sent 17 bytes to user
asgn1_driver: device closed
asgn1_driver: device opened (count=5)
asgn1: SET_MODE called
asgn1_driver: device closed
asgn1_driver: device opened (count=6)
asgn1_driver: received 12 bytes: [[KERNEL] hello-kernel]
asgn1_driver: device closed
asgn1_driver: device opened (count=7)
asgn1_driver: sent 21 bytes to user
asgn1_driver: device closed
asgn1_driver: device opened (count=8)
asgn1: GET_STATS called
asgn1_driver: device closed
asgn1_driver: device opened (count=9)
asgn1: RESET_BUFFER called
asgn1_driver: device closed
asgn1_driver: device opened (count=10)
asgn1_driver: device closed
asgn1_driver: device opened (count=11)
asgn1: GET_STATS called
asgn1_driver: device closed
asgn1_driver: module unloaded
===== TASK 1 FINISHED =====
```

open #	operation	buffer_len	write_count	read_count
1	GET_VERSION	0	0	0
2	SET_MODE 1 (echo)	0	0	0
3	write 17 B	17	1	0
4	read 17 B	17	1	1
5	SET_MODE 0 (kernel)	17	1	1
6	write 12 B (+9 B prefix)	21	2	1
7	read 21 B	21	2	2
8	GET_STATS	21	2	2
9	RESET_BUFFER	0	0	0
10	read (empty)	0	0	0
11	GET_STATS	0	0	0

Figure 5: Driver state after each of the 11 opens in the Task 1 test, taken from the log above

Figure 5 lays the same log out step by step. Each step opens the device once, which is why `open_count` reaches 11. In echo mode the driver stores the 17 bytes unchanged. In kernel mode the user writes 12 bytes, the driver prepends the 9-byte `"[KERNEL] "` prefix and stores 21 bytes; `write()` still returns 12, because its return value must say how many of the *user's* bytes were consumed, otherwise `echo` would try to write the rest again. That is why `GET_STATS` reports `BUFFER LEN 21` but `LAST WRITE SIZE 12`. After `RESET_BUFFER`, `cat` returns nothing (`after reset = []`), and the read count stays 0 because `asgn1_read()` returns end-of-file before it touches the counter when the buffer is empty. The final `GET_STATS` confirms that buffer length, write count and read count are back to zero, while the open count keeps growing since every ioctl itself needs an open. The module then unloads cleanly: the last kernel message is `asgn1_driver: module unloaded`, with no warning or oops in between.

Task 2: test_counter.sh and /proc/counter_info

```
/ # test_counter.sh
===== ASSIGNMENT 01 - TASK 2 TEST =====
counter_driver: loaded
[1] INITIAL STATS
Counter value : 0
Read count    : 0
Write count   : 0
[2] THREE READS - expected: 0, 1, 2
0
1
2
[3] STATS AFTER THREE READS
Counter value : 3
Read count    : 3
Write count   : 0
[4] SET COUNTER = 100
next read:
100
Counter value : 101
```

```
Read count    : 4
Write count   : 1
[5] RESET COUNTER = 0
next read:
0
Counter value : 1
Read count    : 5
Write count   : 2
[6] INVALID NEGATIVE VALUE
negative value rejected correctly
[7] INVALID TEXT VALUE
text value rejected correctly
[8] UNLOAD MODULE
counter_driver: unloaded
[9] DMESG
counter_driver: loaded
counter_driver: unloaded
===== TASK 2 FINISHED =====
```

step	operation	returned	counter	read_count	write_count	/proc snapshot
1	insmod		**0**	0	0	#1
2	read	0	**1**	1	0	
3	read	1	**2**	2	0	
4	read	2	**3**	3	0	#2
5	write "100"		**100**	3	1	
6	read	100	**101**	4	1	#3
7	write "0"		**0**	4	2	
8	read	0	**1**	5	2	#4
9	write "-1"	-EINVAL	**1**	5	2	
10	write "abc"	-EINVAL	**1**	5	2	

Figure 6: Counter state through the Task 2 test; the yellow tags mark the four `/proc/counter_info` snapshots

`/proc/counter_info` was read at four points: before any write (0 / 0 / 0, and 3 / 3 / 0 after the three reads), after writing 100 (101 / 4 / 1) and after the reset to 0 (1 / 5 / 2). The values follow the specification exactly. Each `cat` opens the device anew, so `*ppos` starts at 0, the driver returns one value, and the second `read()` of the same `cat` gets 0 and stops; three `cat` commands therefore print 0, 1, 2 and leave the counter at 3. Writing `"100\n"` sets the counter to 100 (`kstrtoint()` accepts the trailing newline from `echo`), the next read returns 100 and moves it to 101. Writing 0 is the reset, and the next read returns 0. Writing `-1` and `abc` both return `-EINVAL`, which makes `echo` fail, and the counter and write count stay unchanged. The kernel log contains only the load and unload messages, as this driver logs nothing on normal reads and writes.

Issues found during testing and review

The first point comes from the Task 1 log. `RESET_BUFFER` clears the buffer and resets the read and write counters, but `LAST WRITE SIZE` is still 12 after the reset. The assignment only asks for the buffer and the two counters to be cleared, so this is not wrong, but it is inconsistent: after a reset there is no "last write" any more. We kept the behaviour and documented it; a one-line `last_write_size = 0;` in the reset branch would make the statistics fully consistent.

The second point is the read–increment race in `counter_read()` described in section 4.2 and Figure 3. It was found by reading the code rather than by a test, because it needs two readers at exactly the same time. The way we reasoned about it is the same way one would debug it with `dmesg`: write down, for each CPU, the order of the shared-memory operations, and check whether any interleaving produces a result that no single reader could have produced. Here one does, and the fix is `atomic_fetch_inc()`.

The kernel log was our main debugging tool throughout. Task 1 prints one line per open, read, write, close and ioctl, so each line of the script output can be matched to what the driver did; for example, the pair "received 12 bytes" and "sent 21 bytes to user" is what made the difference between bytes consumed and bytes stored in kernel mode obvious.

6. Theory questions

Question 1. Module reference counting

Each loaded module has a reference count, visible in the "Used by" column of `lsmod`. It goes up whenever something takes a reference with `try_module_get()`/`__module_get()` and down with `module_put()`. For a driver the most common source is opening its device: because `asgn1_fops.owner = THIS_MODULE`, the VFS calls `fops_get()`, which takes a reference on our module before calling `asgn1_open()`, and drops it in `__fput()` after `asgn1_release()` when the last file descriptor is closed. Other modules that use our exported symbols also hold references.

If `rmmod asgn1_driver` is run while a process still holds `/dev/asgn1` open, the `delete_module` system call sees a non-zero count and refuses with `EWOULDBLOCK` (BusyBox prints "can't unload module: Resource temporarily unavailable"). This is what protects the kernel: if the module were freed, the open file would still point at `asgn1_fops`, and the next `read()` would jump into freed memory.

The correct way to unload is to make the count drop: find the process holding the file (`fuser /dev/asgn1` or `lsof`), close or kill it, and `rmmod` again. There is also `rmmod -f`, which works only if the kernel was built with `CONFIG_MODULE_FORCE_UNLOAD`; it removes the module anyway and leaves exactly the dangling pointer described above, so it is a last resort for development, never for a product.

Question 2. Why copy_to_user() and not memcpy()

The `arg` of `ASGN1_GET_STATS` is an address in the calling process's virtual address space. On our 32-bit ARM kernel the lower 3 GB belong to the current process and the upper 1 GB to the kernel, so the kernel could in principle dereference it directly, and that is exactly the danger. The pointer comes from user space and may be invalid, may point to a page that is not present yet, or may point into kernel memory on purpose.

`copy_to_user()` first checks with `access_ok()` that the whole range `[arg, arg+20)` lies below the user/kernel boundary, so a malicious pointer cannot make the driver overwrite kernel

data. The copy itself is done by instructions listed in the kernel's exception table. If one of them triggers a page fault, the MMU raises a data abort; the fault handler sees that the faulting address is in the exception table and either maps the page (if it is a valid user page that was just not loaded yet) and lets the copy continue, or jumps to a fixup that makes `copy_to_user()` return the number of bytes not copied, which we turn into `-EFAULT`.

With `memcpy()` none of this happens. A user page that is not present causes a data abort from an instruction the fault handler does not know, so the kernel prints "Unable to handle kernel paging request" and oopses. In addition, ARMv7 kernels like ours are built with `CONFIG_CPU_SW_DOMAIN_PAN`: the user-space domain is set to "no access" while the kernel runs, and only the uaccess helpers open it briefly. A plain `memcpy()` to a user address then raises a domain (permission) fault even when the page is present, so it fails immediately instead of silently working on some machines.

Question 3. MTD vs the block layer

The block layer assumes a device made of sectors that can each be read and overwritten in place, in any order, an unlimited number of times. MTD exposes raw flash as it really is: a page can be programmed only after its whole erase block has been erased, erase blocks are much larger than pages, each block survives a limited number of erase cycles, NAND comes with bad blocks and needs ECC and an out-of-band area. Accordingly, an MTD device offers read, write and an explicit erase operation, and it reports erase size, page size and OOB size (as `mtdinfo` showed in LAB-02); it does not offer sector overwrite.

A storage device needs MTD when it is raw flash without its own controller: parallel or SPI NOR (the boot flash, and the two NOR banks that QEMU emulates for vexpress at `0x40000000` and `0x44000000`, visible in our boot log), and raw NAND chips connected directly to the SoC's NAND controller. On these the filesystem itself (JFFS2, UBIFS on UBI) must handle erasing, wear leveling and bad blocks. Managed flash such as eMMC, SD cards or USB sticks has a controller with an FTL inside that already does all of that and presents sectors, so it is a normal block device (`mmcblk0`, `sda`) and runs ext4 directly.

Question 4. *ppos in counter_read()

`cat` does not know how big a file is; it calls `read(fd, buf, 4096)` in a loop and stops only when `read()` returns 0 (end of file) or an error. In the kernel each call goes `ksys_read()` → `vfs_read()` → `file->f_op->read(file, buf, count, &pos)`, where `pos` is a copy of `file->f_pos`; after the driver returns, `f_pos` is updated from it. The file position is therefore the only memory a driver has between two `read()` calls of the same open.

Our driver uses it as follows: if `*ppos != 0` it returns 0 straight away; otherwise it formats the value, copies it, increments the counter and adds the length to `*ppos`. The first `read()` of a `cat` returns, for example, `"5\n"` (2 bytes) and moves `f_pos` to 2; the second sees a non-zero position, returns 0, and `cat` exits. That is why each `cat` in the test printed exactly one number.

Without the check, every call would return a fresh value and increment the counter. The second `read()` would return `"6\n"`, the third `"7\n"`, and so on; `read()` would never return 0, so `cat` would print an endless rising sequence of numbers and the counter would race upward until the user pressed Ctrl+C. The stop condition is always the same: a `read()` returning 0, which the driver can only decide correctly by looking at the file position.